Module 13: State Management

Managing Application State in Angular 21



Module Overview

Learning Objectives

01

Understand State

Learn what application state is and why it matters

02

Differentiate State Types

Distinguish between component state and shared state

03

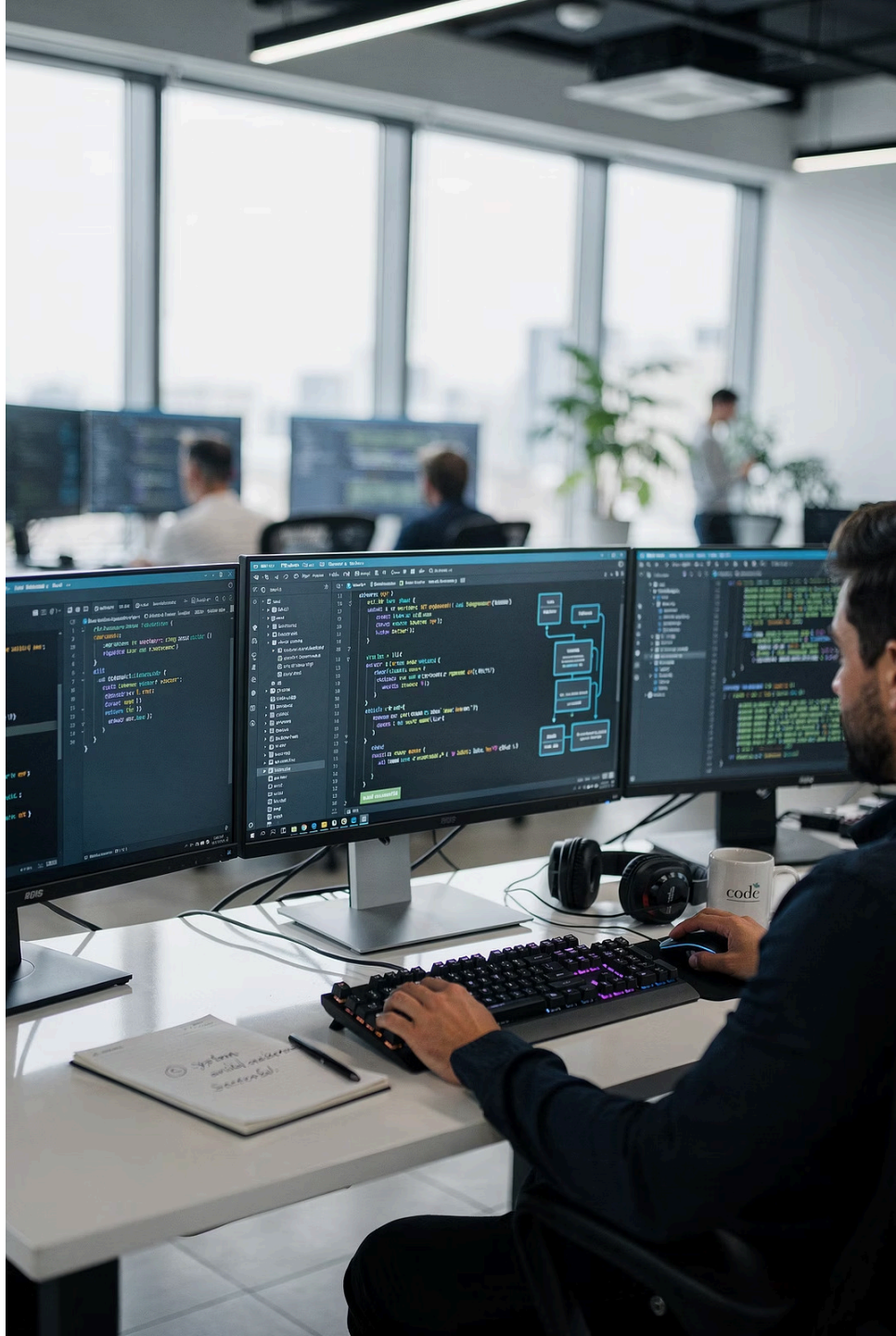
Manage with Services & Signals

Use services and Angular Signals for reactive state

04

Choose the Right Approach

Apply store patterns and select the best strategy per scenario



What is State?

State represents the data that determines how an application behaves and what it displays. Whenever application data changes, **the UI must reflect the latest state**.



Logged-in User

Who is authenticated and their permissions



Shopping Cart

Items selected for purchase



Employee List

Records loaded from the server



Notifications

Alerts and messages for the user



Dashboard Metrics

Aggregated data for analytics views

Types of State

Angular applications manage several distinct categories of state. Always choose **the smallest scope** that satisfies the business requirement.

State Type	Example	Description	Scope
Component State	Form values, selected tab	Owned by a single component	Local
Shared State	Logged-in user, theme	Used across multiple components	Global
Server State	Employee records, products	Fetched from a backend API	Remote
UI State	Dialog visibility, loading spinner	Controls display behavior	Local / Shared
Session State	User preferences	Persisted for the user session	User-scoped



Component State

Component state belongs to a **single component** and is never shared with others. It is the simplest and fastest form of state to work with.

Common Examples

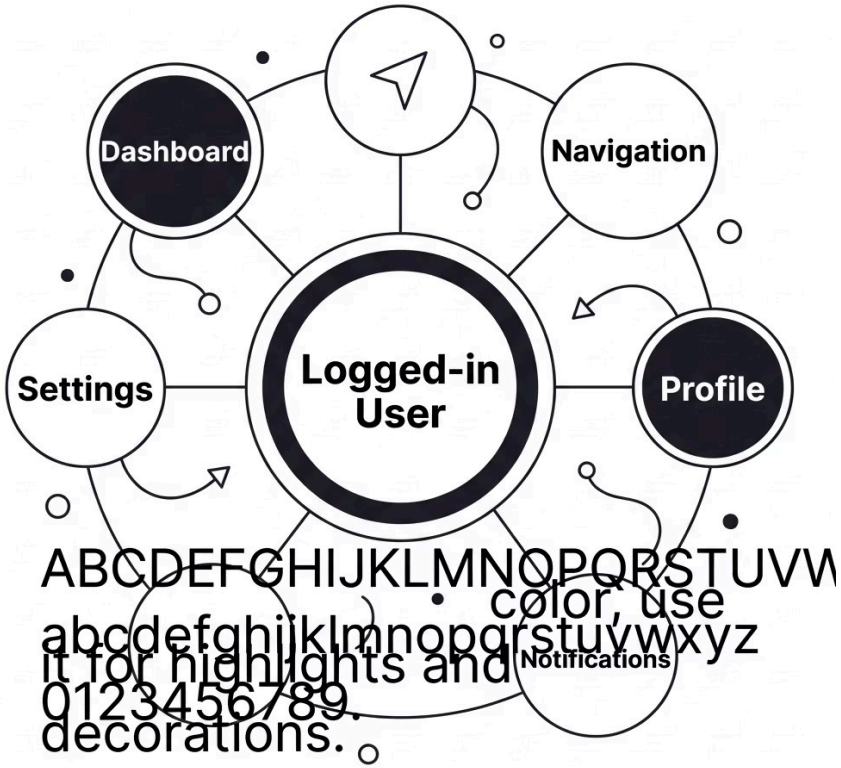
- Form data and search text
- Selected tab or current page
- Expanded or collapsed panel

Benefits

Simple	**Fast**	**Easy to Maintain**	**Limited Scope**

Shared Application State

Some data must be accessible across many components simultaneously. Shared state provides a **single source of truth** to keep the entire application consistent.



Common Shared State

- Authentication & user identity
- Theme and language preferences
- Notification feed
- Shopping cart contents

A single source of truth prevents inconsistencies when the same data is displayed in multiple places.



Service-Based State Management

Angular services act as a **centralized holder** for shared state. Components inject the service and read or update state through its public API — no duplication, no drift.

1

Components

Inject the service and react to state changes

2

Shared Service

Holds and exposes application state

3

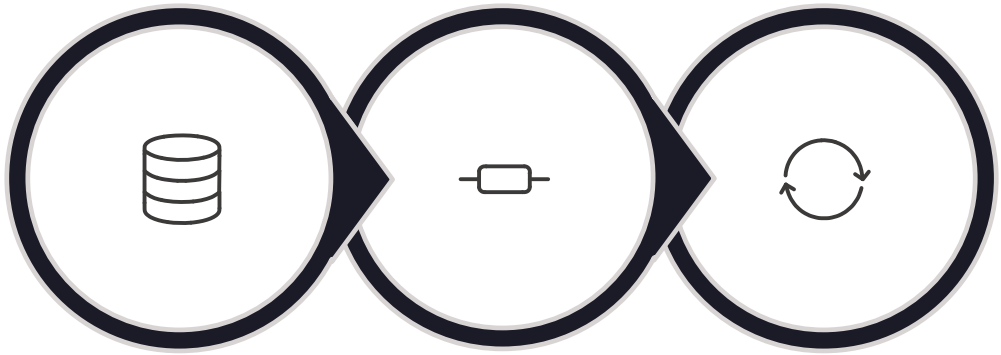
Application State

Single, consistent source of truth

Best for: Shared user information, dashboard data, and application settings in small-to-medium apps.

Signal-Based State Management

Introduced in Angular 16 and matured in Angular 21, **Signals** offer a reactive, fine-grained way to manage state with automatic UI updates and minimal boilerplate.



Advantages

- Reactive by default — no manual subscriptions
- Automatic, precise UI updates
- Minimal boilerplate code
- Better rendering performance

Common Use Cases

Theme	Cart	Filters	Dashboard



Store Design Patterns

A **Store** centralizes both state and business logic in one place, enforcing a predictable, unidirectional data flow across the application.

Store State

Holds the authoritative application data

Update State

Processes actions and mutates state safely

Expose State

Provides reactive streams components subscribe to

Coordinate Logic

Keeps business rules out of components

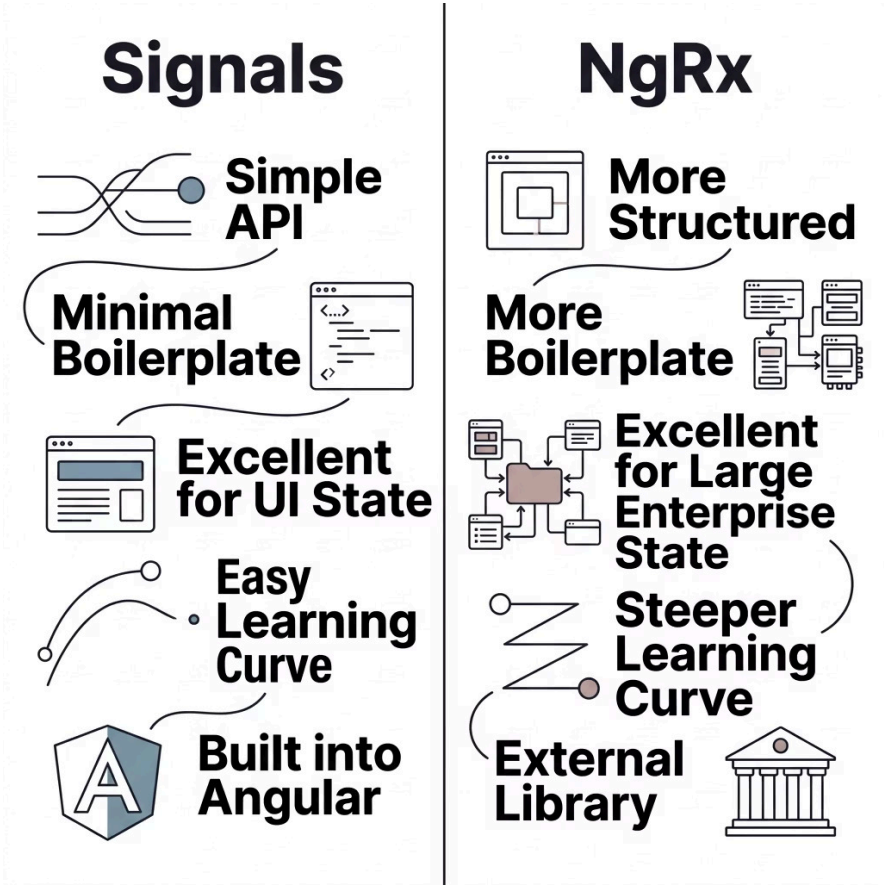
Choosing a State Management Strategy

Match your strategy to the complexity and scope of the state. **Keep state as close as possible to where it is needed.**

Scenario	Recommended Approach
Local component state	Component variables / Signals
Shared UI state	Signals
Shared business state	Service + Signals
Large enterprise state	NgRx / Signal Store
Server / remote data	HttpClient + Service

Signals vs. NgRx

Both are powerful — the right choice depends on your application's scale and team complexity.



Recommendation

Use Signals

For most applications — reactive, lightweight, and built into Angular 21

Use NgRx

Only when enterprise complexity demands advanced patterns like effects, selectors, and action tracing

State Management Best Practices

- [ ] **Keep state minimal**

  Only store what cannot be derived from other state

- [ ] **Avoid duplicate state**

  A single source of truth prevents bugs and inconsistency

- [ ] **Separate UI state from business state**

  Dialog visibility ≠ user authentication data

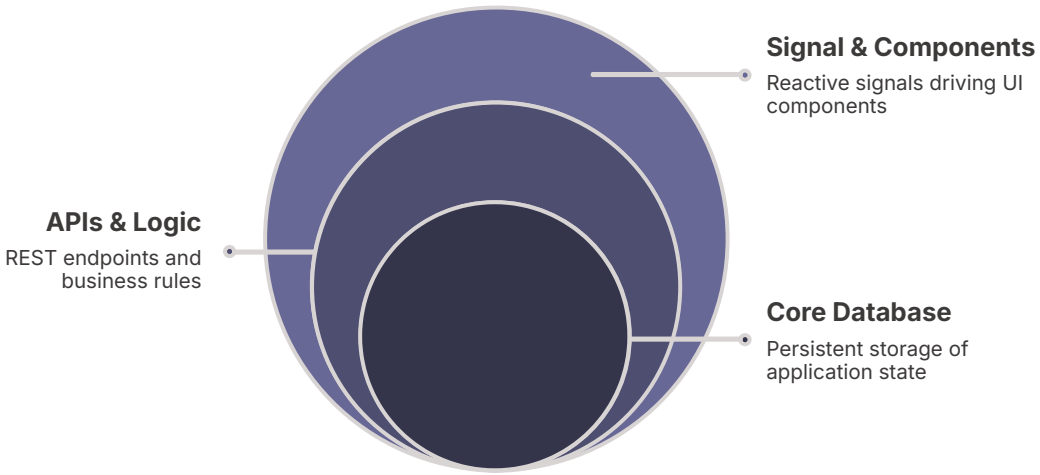
- [ ] **Prefer Signals & Services**

  Introduce NgRx only when application complexity genuinely justifies it



Enterprise State Architecture

At scale, a layered architecture separates concerns cleanly — from the UI all the way to the database.



Why This Architecture Works

Separation of Concerns

Each layer has one clear responsibility

Reactive UI Updates

Signals propagate changes instantly through the stack

Reusable Business Logic

Services and stores are independent of the view layer

Scalable by Design

New features plug in without restructuring the entire app

Module Summary

You now have a complete picture of state management in Angular 21 — from simple component variables to enterprise-grade stores.

Component State

Local, fast, scoped to a single component

Shared & Service State

Single source of truth across multiple components

Signals

Reactive, built-in, and ideal for most use cases

Store Patterns

Centralized, predictable, enterprise-ready

Next Module

Angular Material & Performance Optimization — Angular Material components, OnPush Change Detection, Deferred Loading, and Rendering Optimization.